



Operating System Capstone

Lab 3: Memory Allocator

Sean, Hsien



Overview

You have 2 exercises on this lab

- Exercise 3-1: The Buddy Allocator
- Exercise 3-2: Memory Reservation



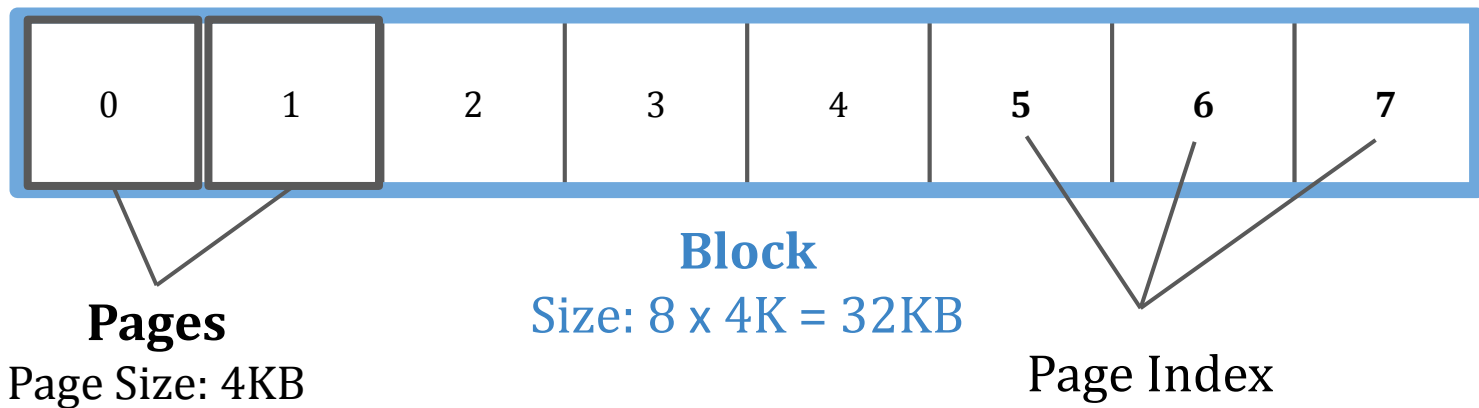
Exercise 3-1: Buddy System

1. **Search mechanism** : Locate the lowest available block in the `free_area` array starting from target level $k \geq \text{order}$
2. **Split logic** : Bisect blocks iteratively when $k > \text{order}$
3. Downgrade and insert unallocated split remnants into the corresponding **free_area** lists



Buddy System

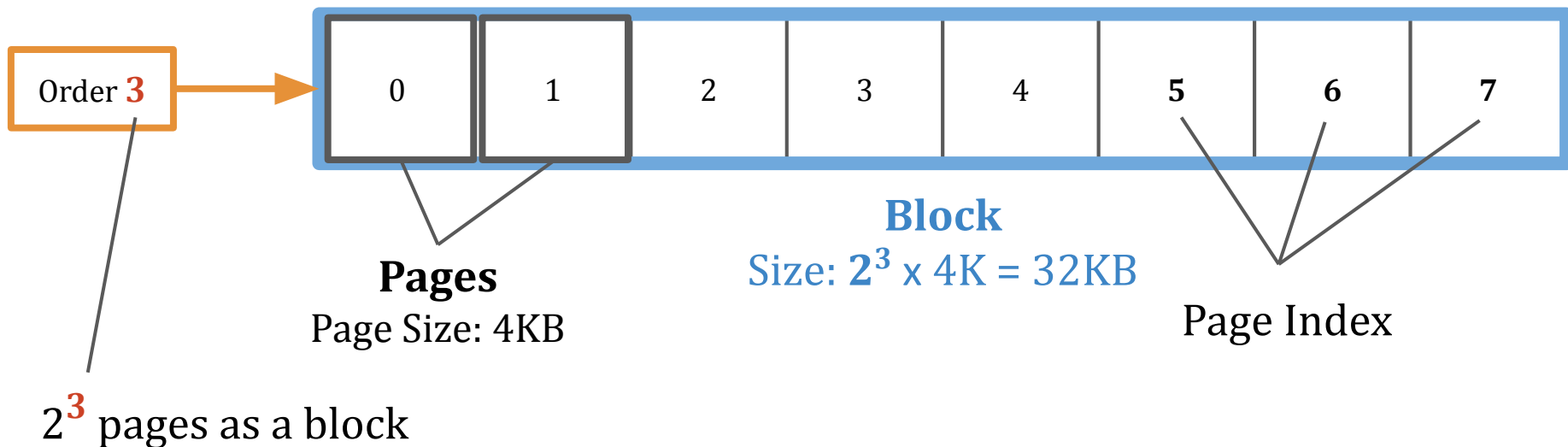
Buddy system is a well-known and simple algorithm for allocating contiguous memory blocks.





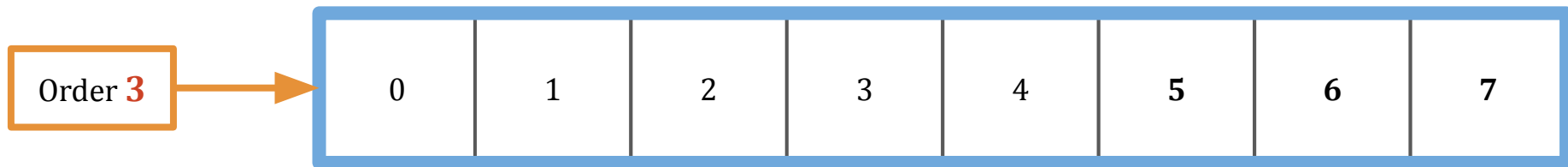
Buddy System

Buddy system is a well-known and simple algorithm for allocating contiguous memory blocks.





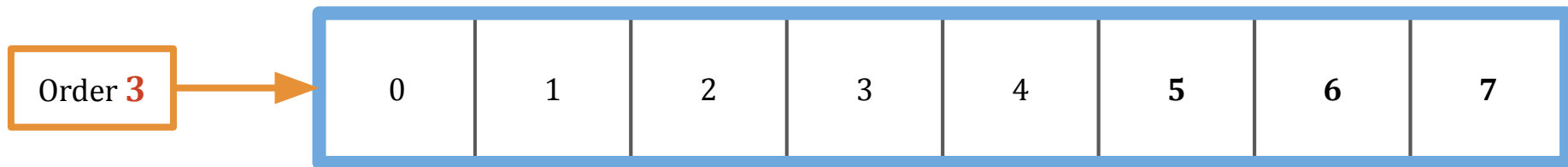
Buddy System — Example



```
void example() {  
    p1 = allocate(2048);  
    p2 = allocate(6144);  
    free(p1);  
    free(p2);  
}
```



Buddy System — Example Allocate 2KB



```
void example() {  
    p1 = allocate(2048);  
    p2 = allocate(6144);  
    free(p1);  
    free(p2);  
}
```

To allocate 2KB:

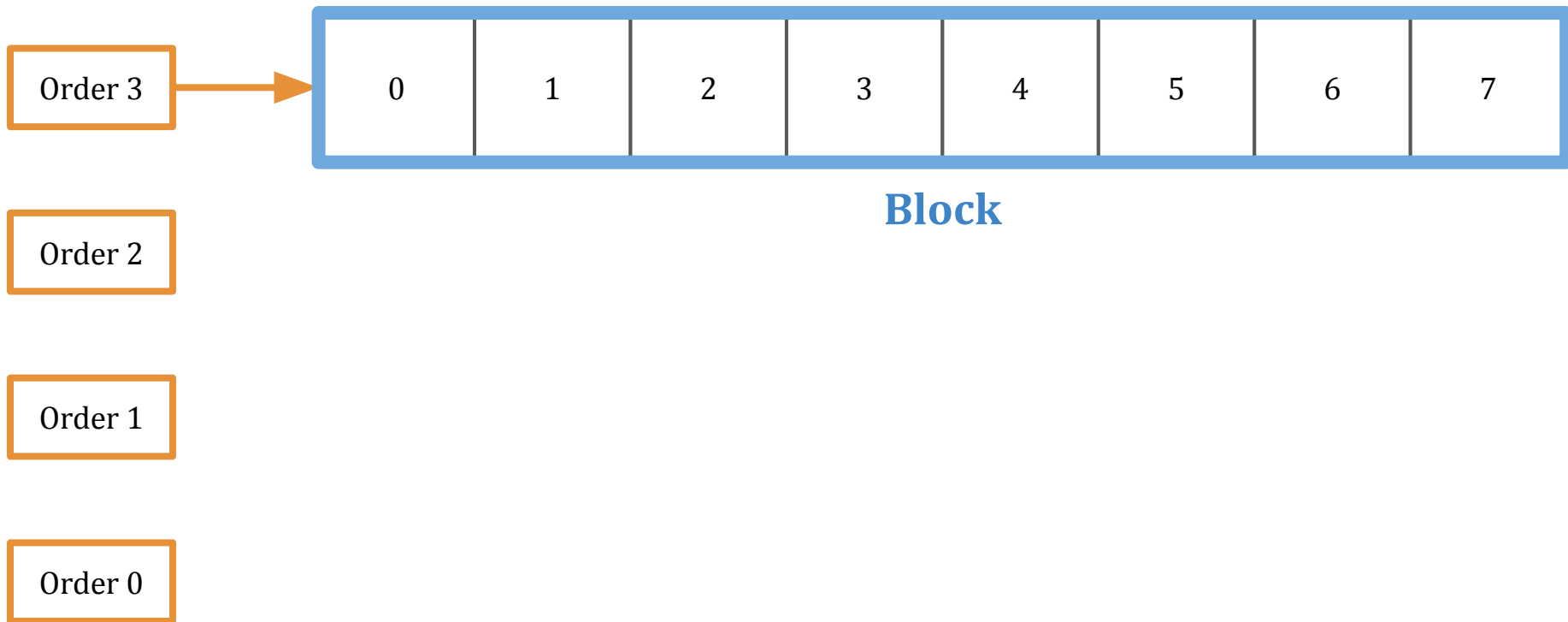
$2\text{KB} \leq 4\text{KB}$ (page size)

$\Rightarrow 1 \text{ page} = 2^0 \text{ page}$

$\Rightarrow$ Allocate a block at order 0

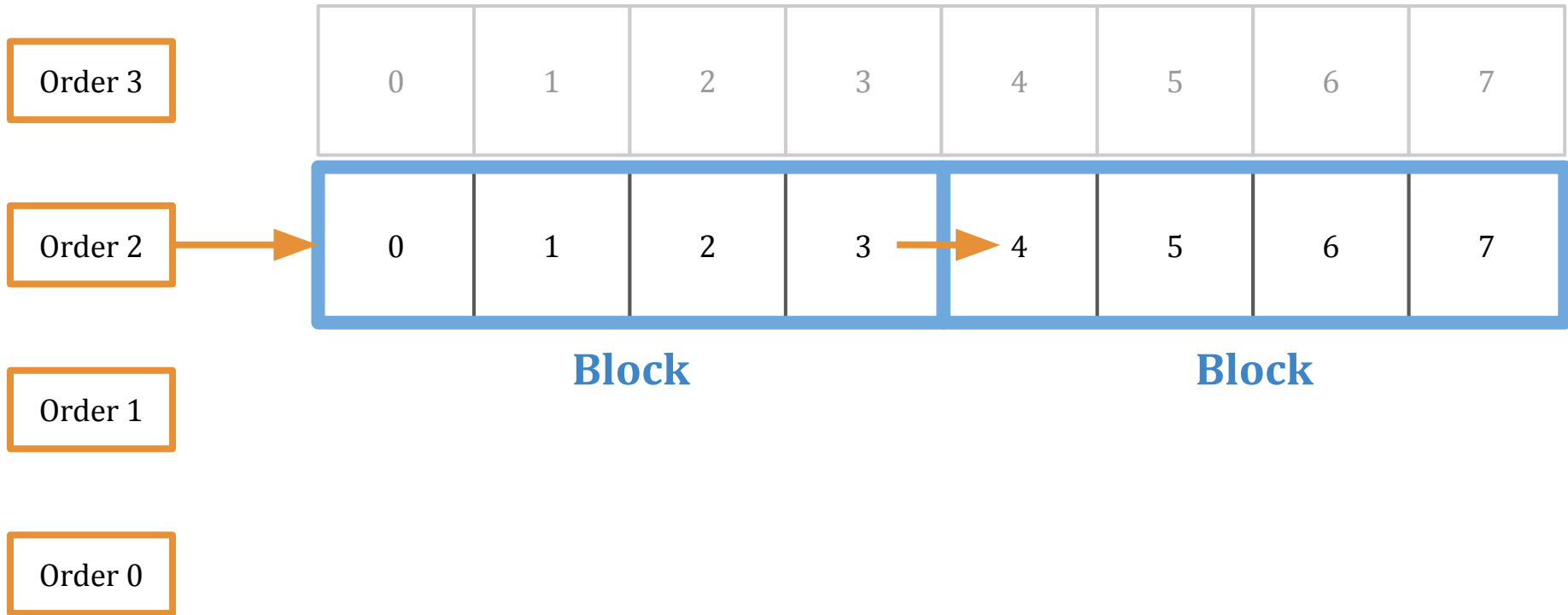


Buddy System — Example Allocate 2KB



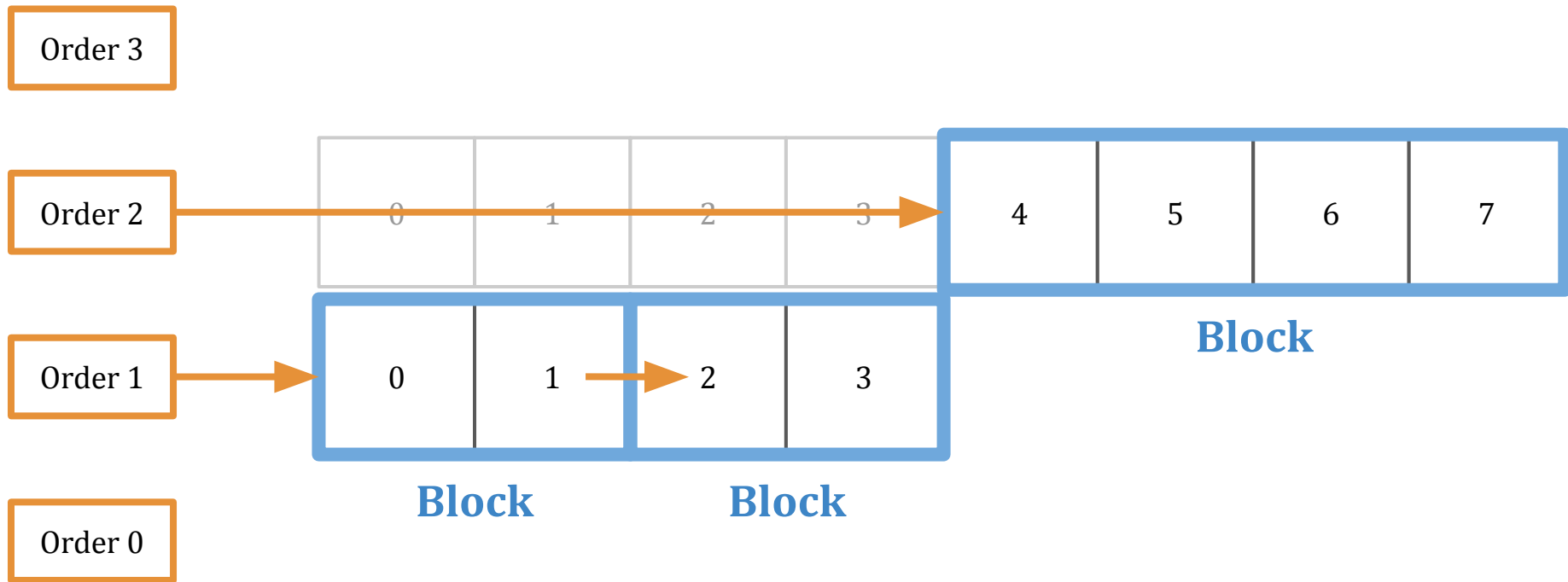


Buddy System — Example Allocate 2KB



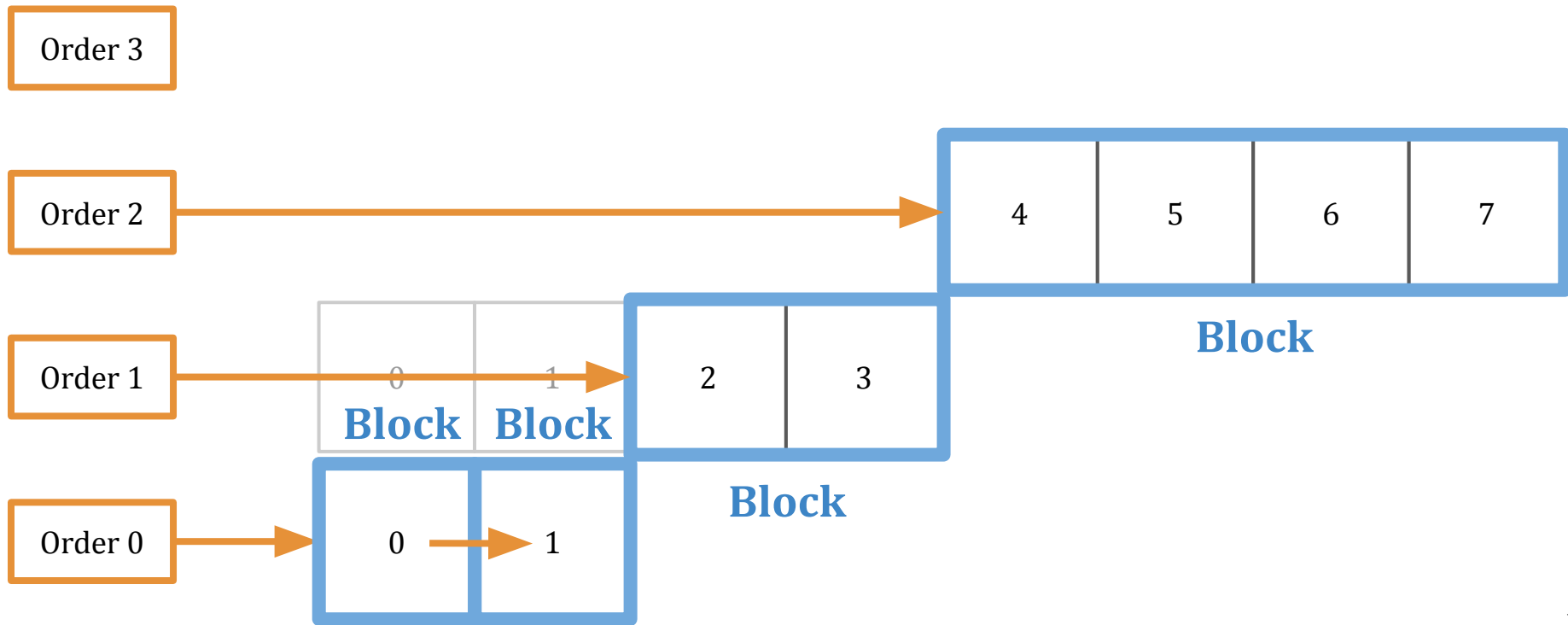


Buddy System — Example Allocate 2KB



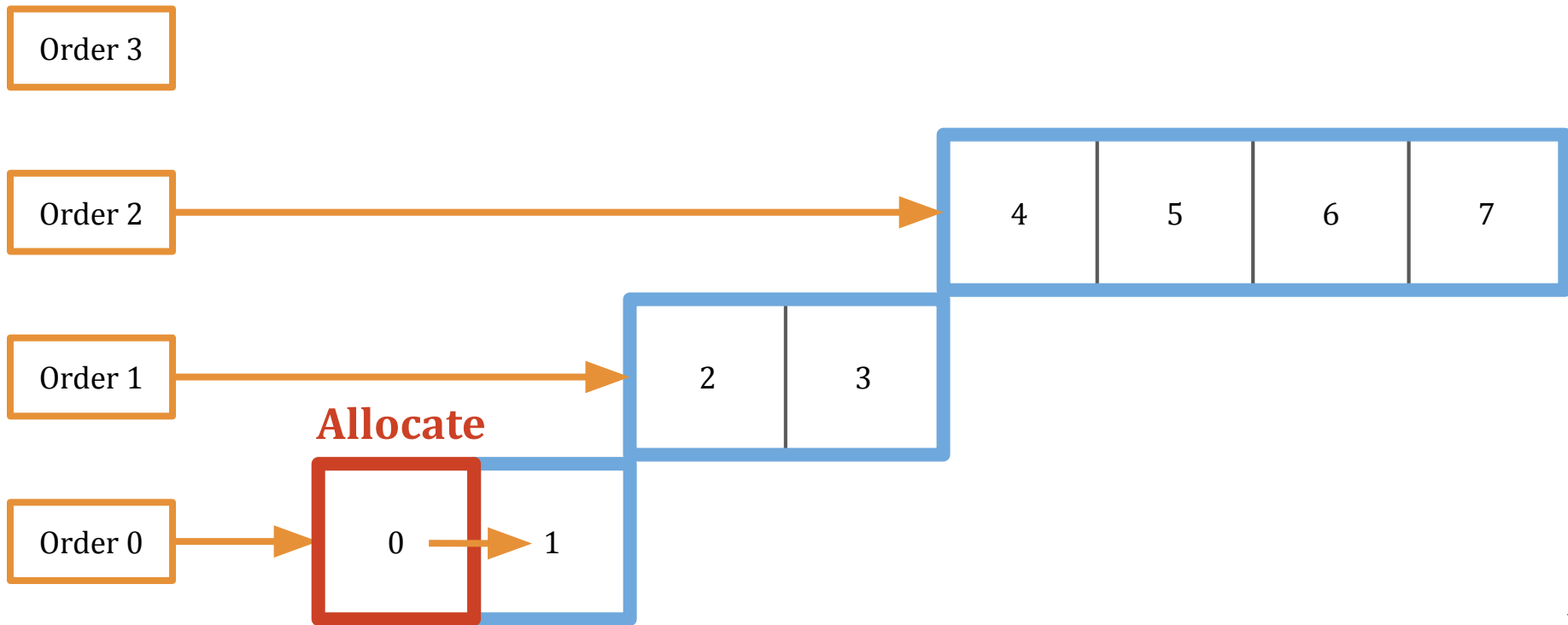


Buddy System — Example Allocate 2KB



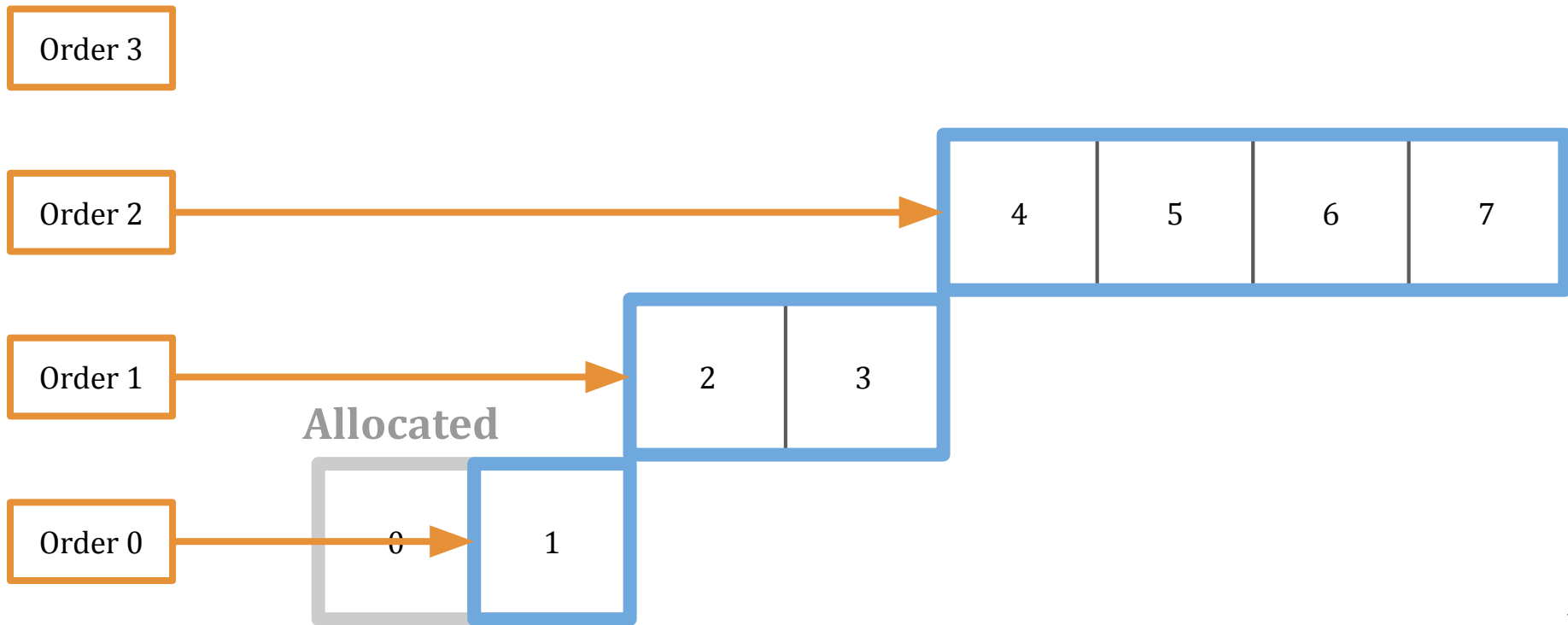


Buddy System — Example Allocate 2KB



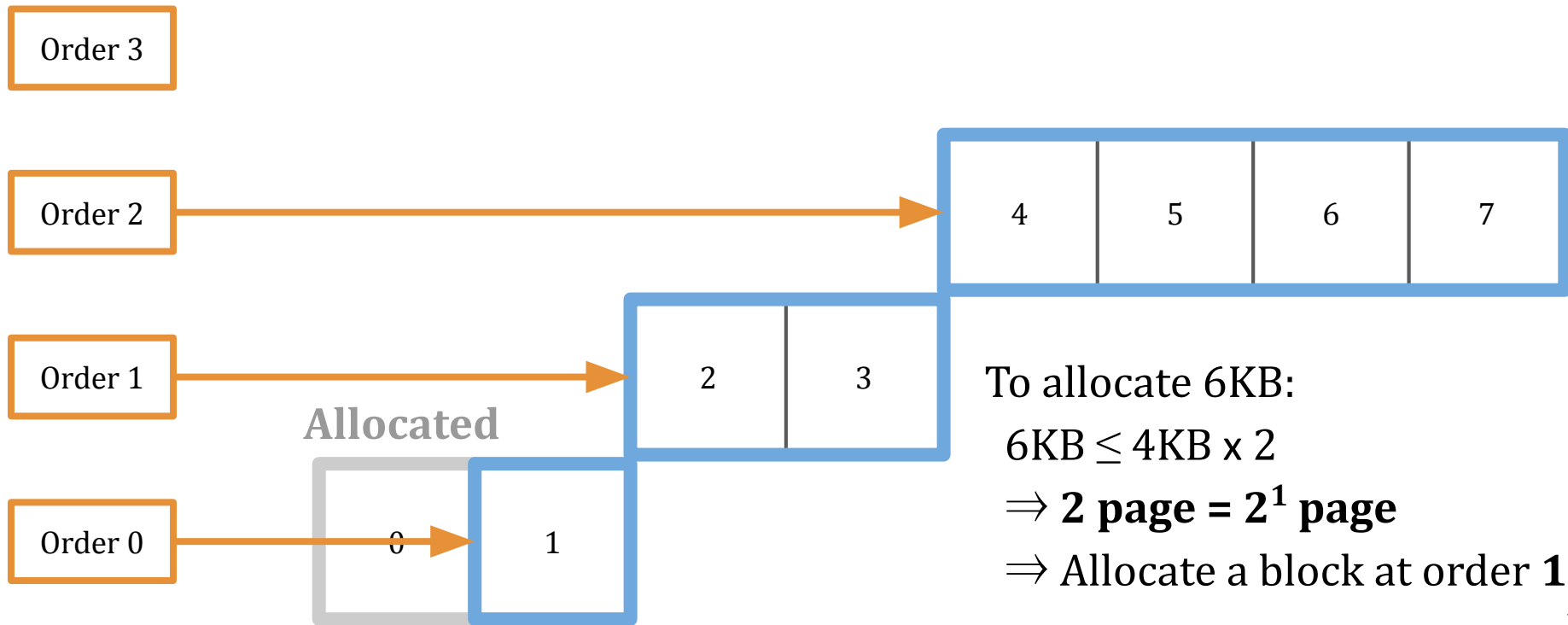


Buddy System — Example Allocate 2KB



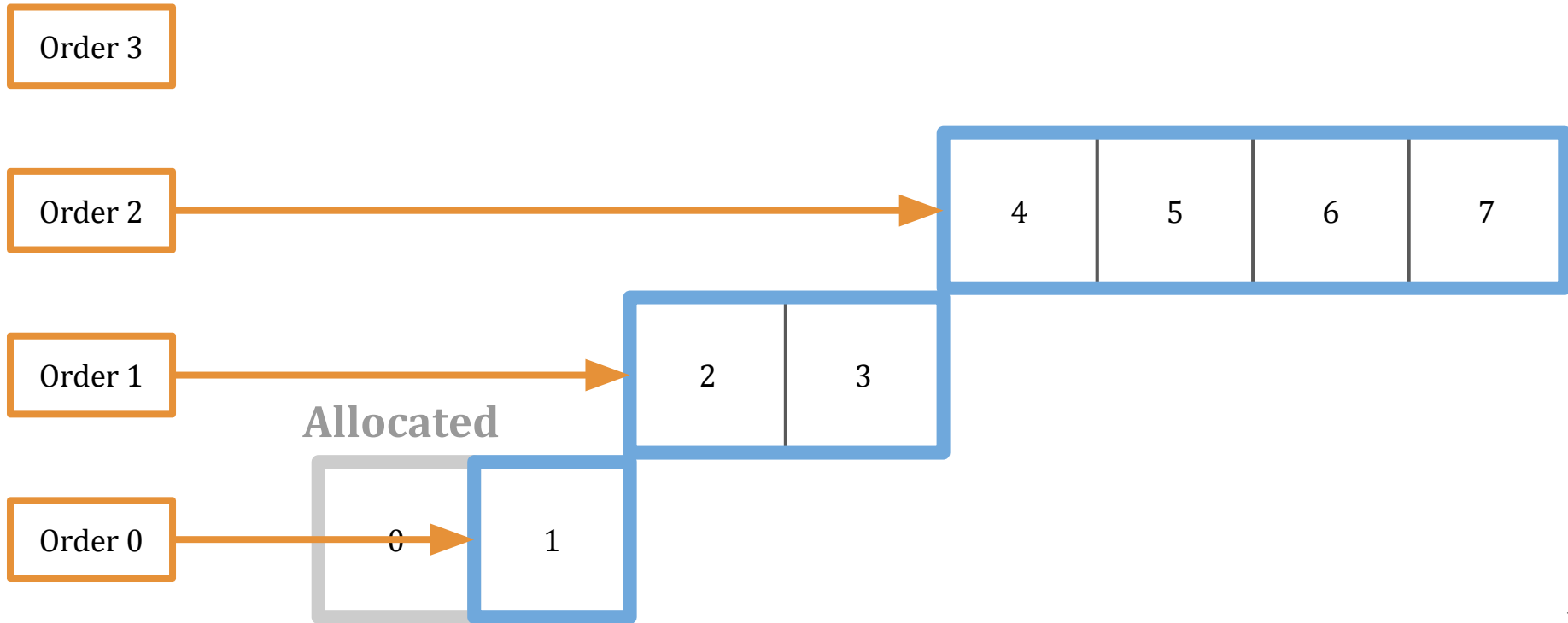


Buddy System — Example Allocate 6KB



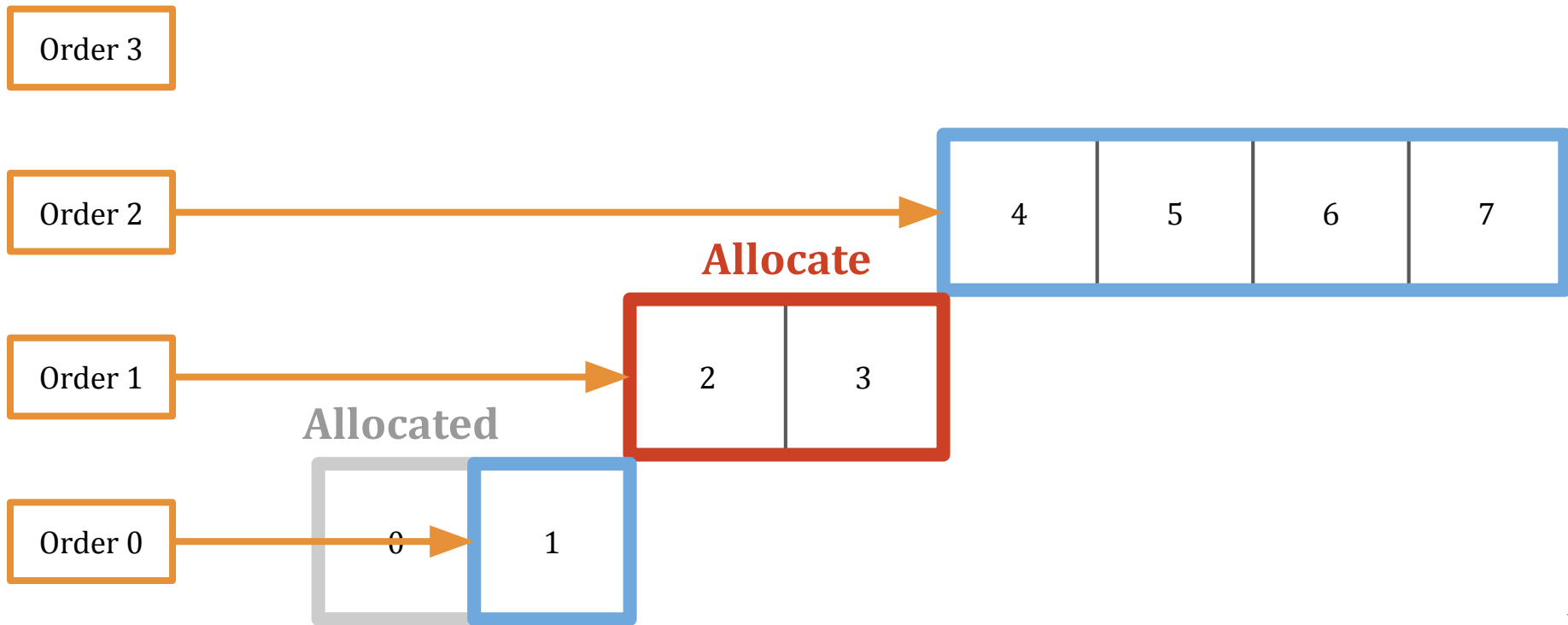


Buddy System — Example Allocate 6KB



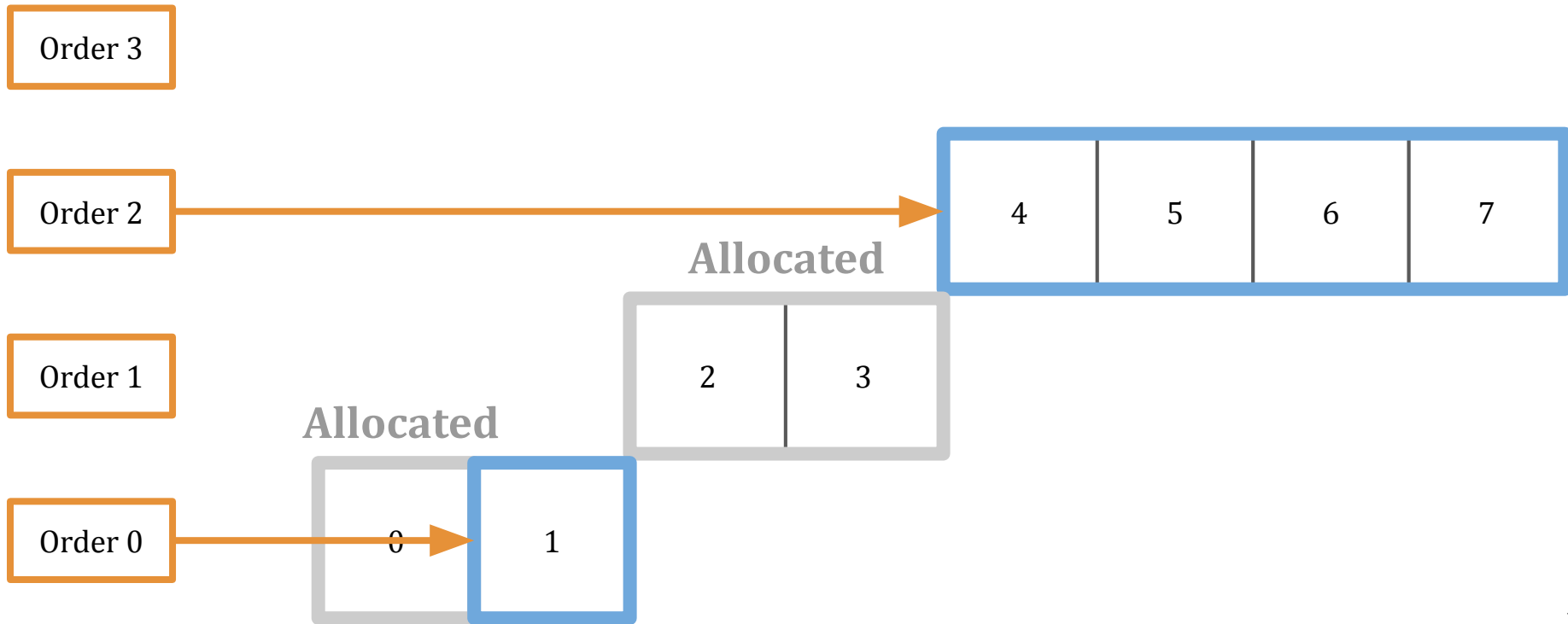


Buddy System — Example Allocate 6KB



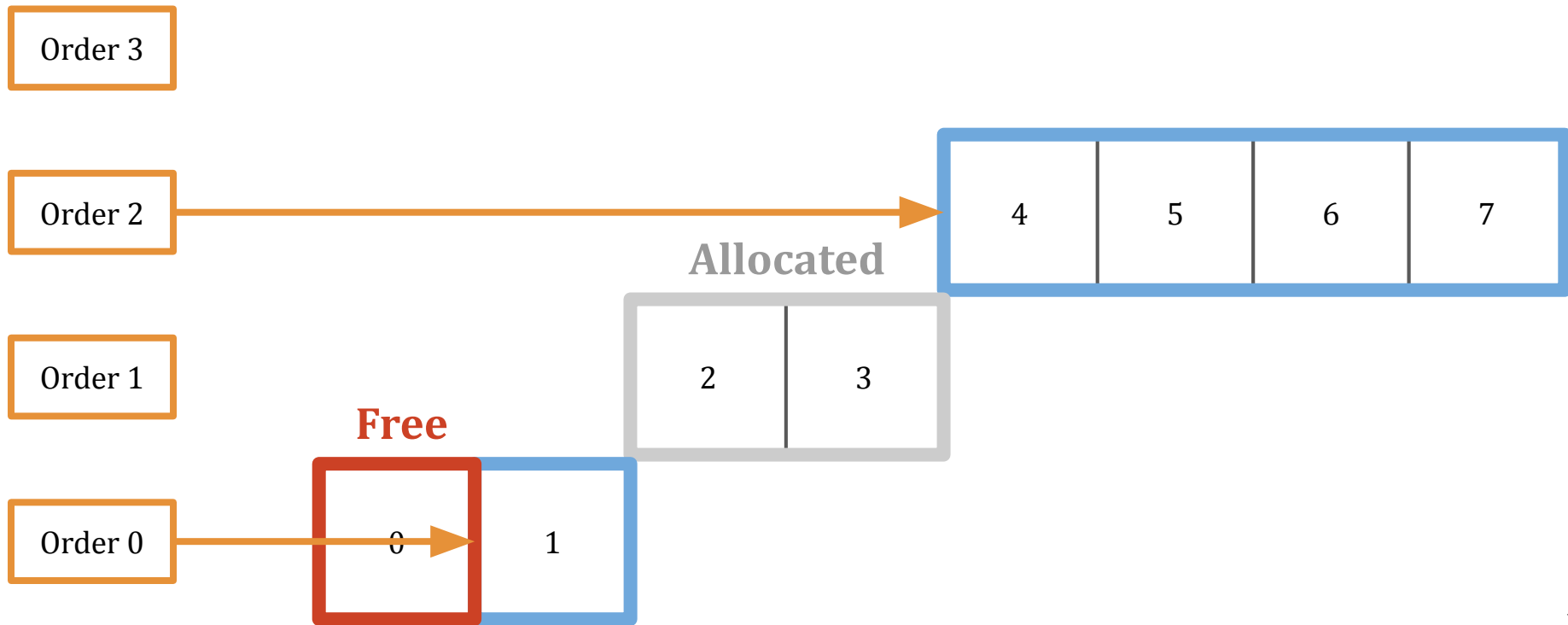


Buddy System — Example Allocate 6KB



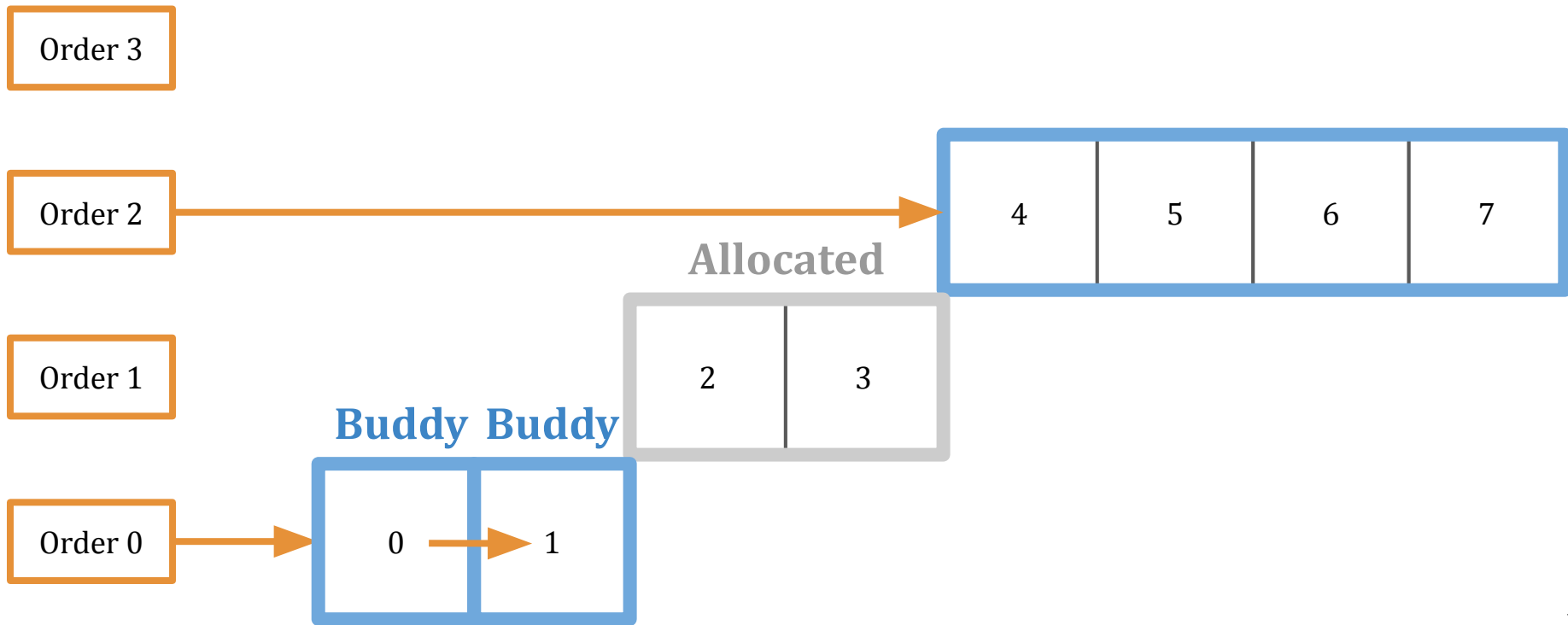


Buddy System — Example Free 2KB



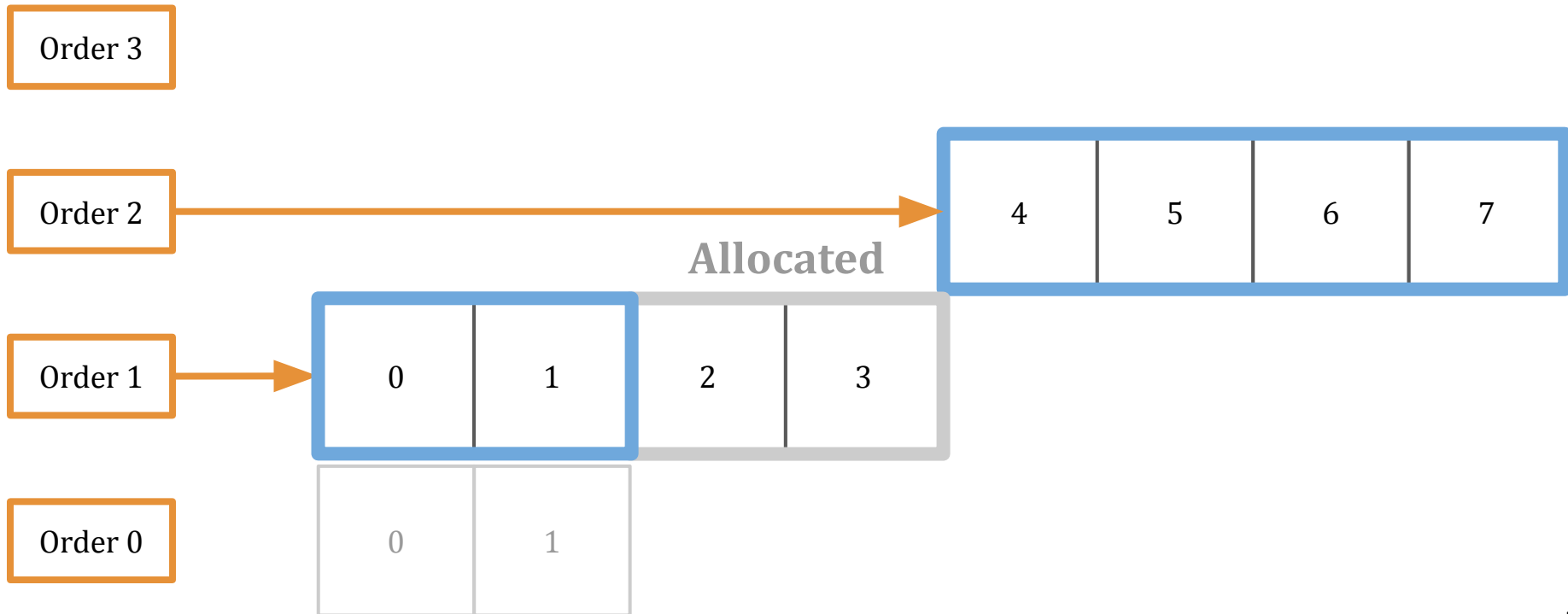


Buddy System — Example Free 2KB



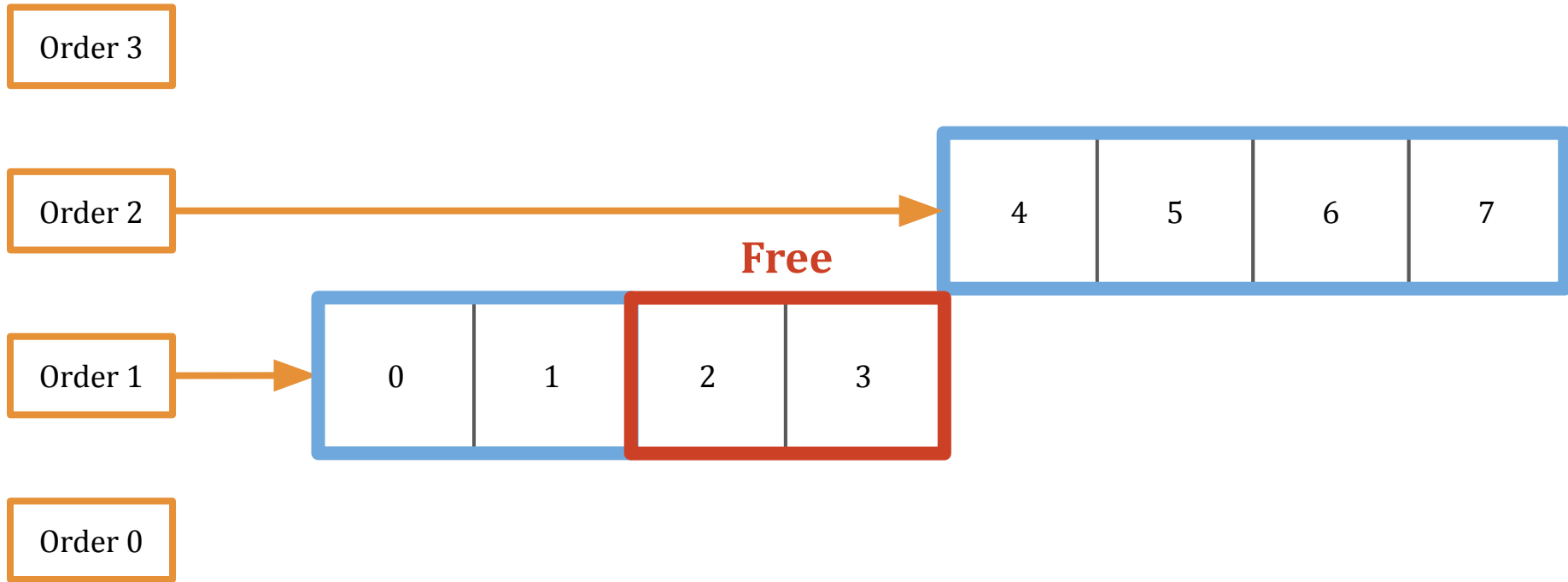


Buddy System — Example Free 2KB



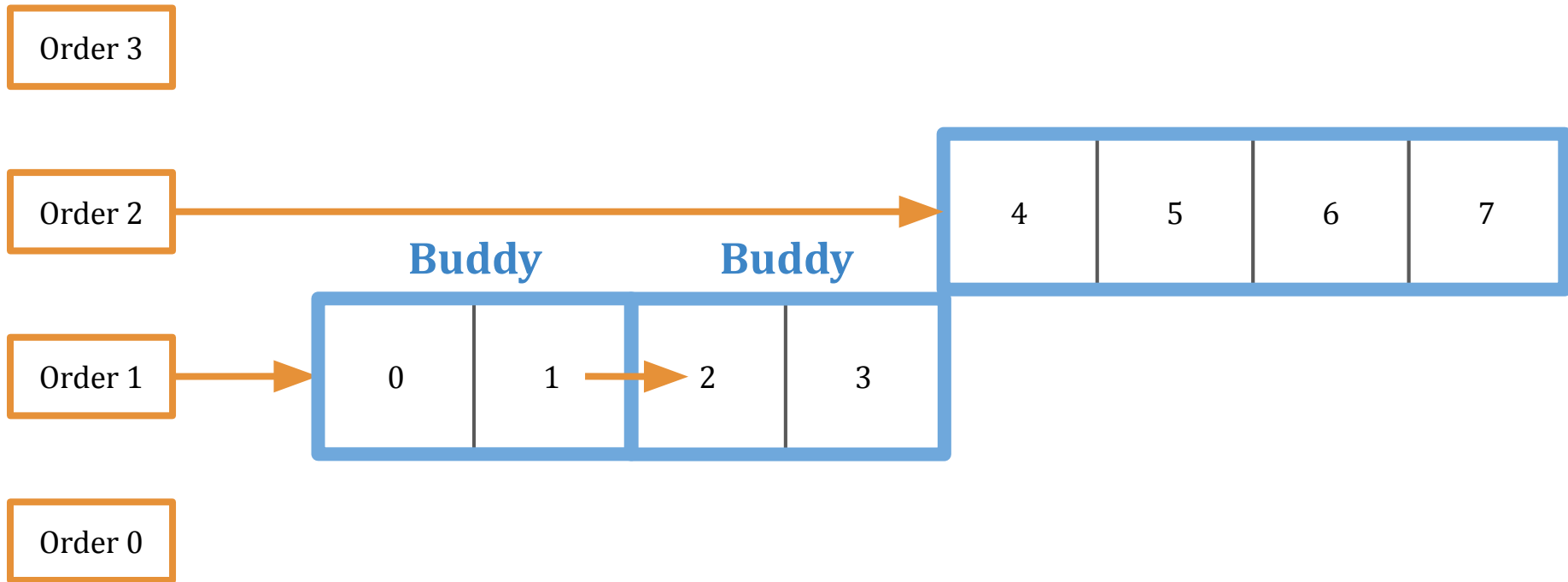


Buddy System — Example Free 6KB



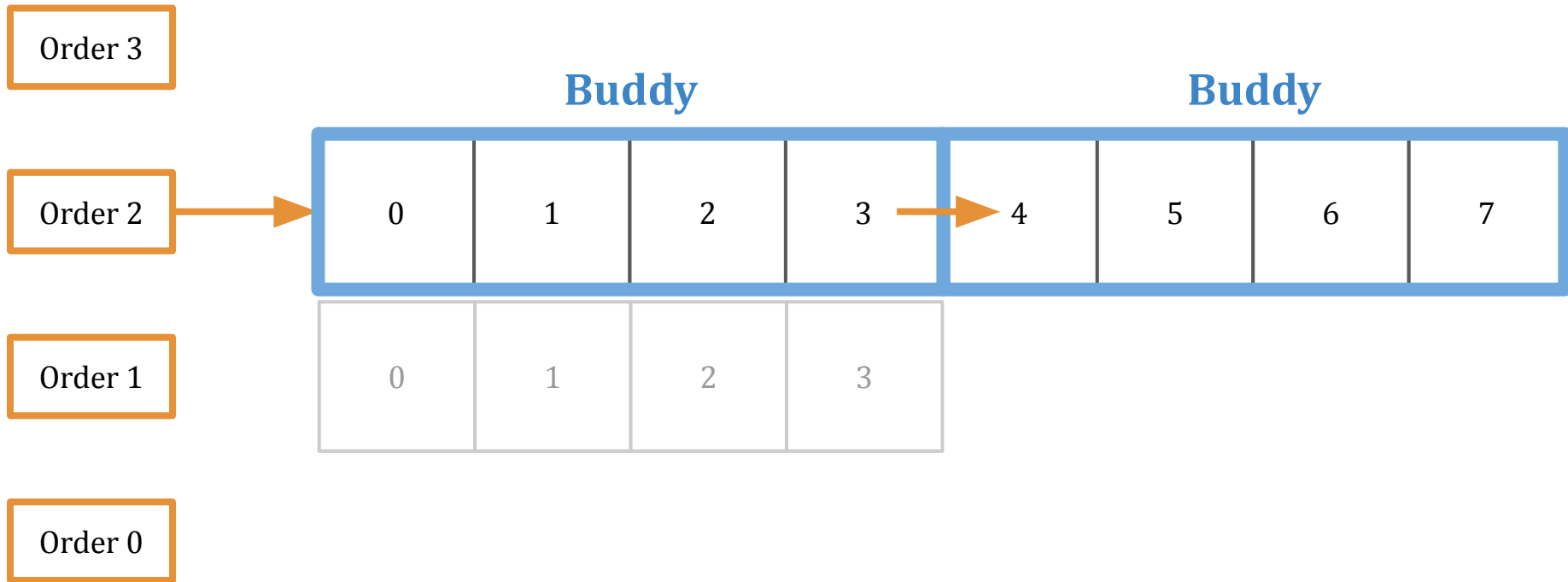


Buddy System — Example Free 6KB



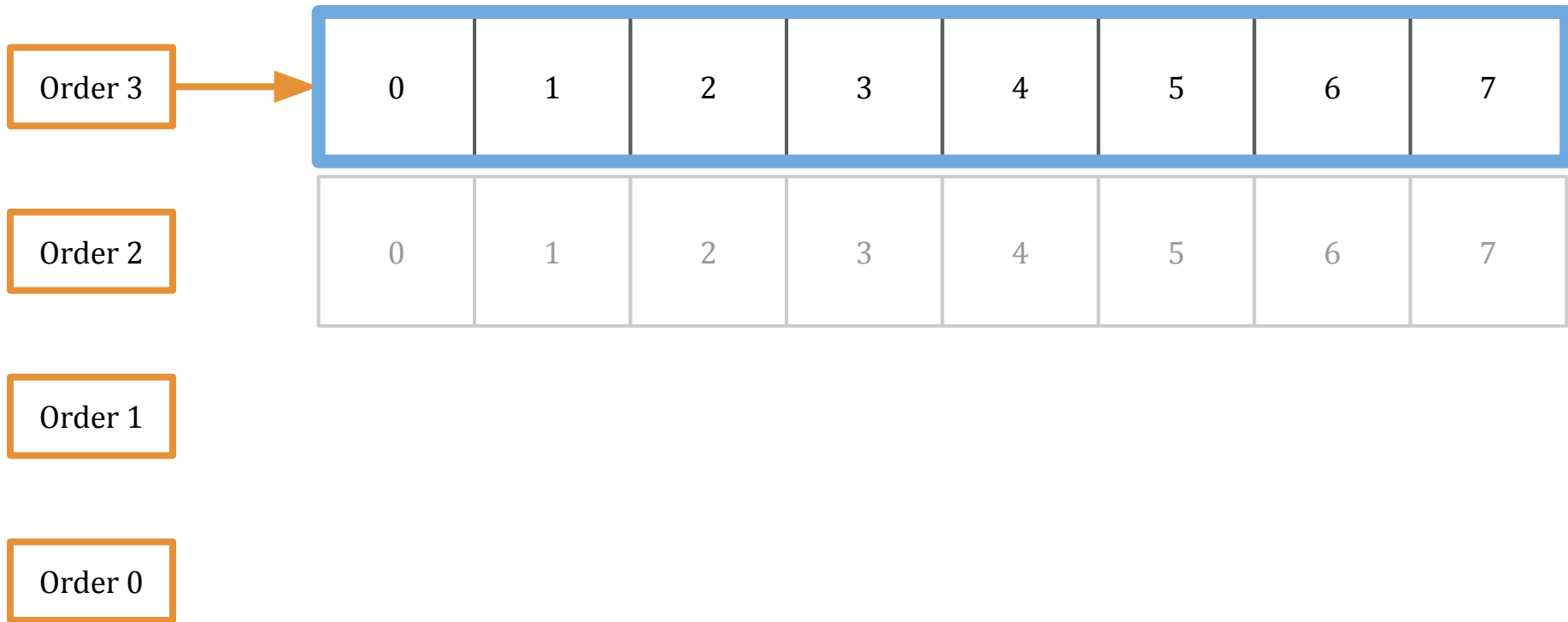


Buddy System — Example Free 6KB



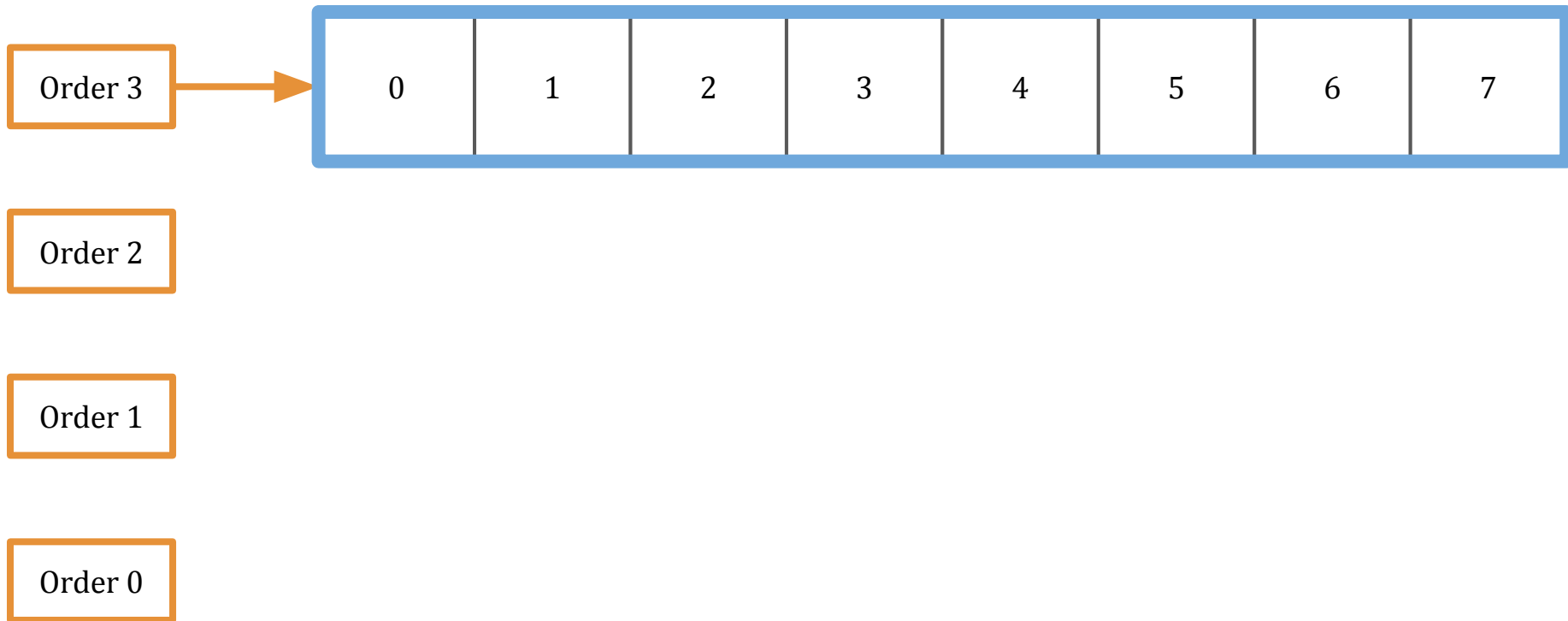


Buddy System — Example Free 6KB





Buddy System — Example Free 6KB





Exercise 3-1: Implementation Logic

- **alloc_pages :**
 - Detach the leading block from the identified “free_area” list
 - Execute iterative bisection (split) if the located block's order exceeds the requested order
 - Append the residual split halves into the “free_area” lists
- **free_pages :**
 - Revert the target block's metadata to the unallocated state
 - Merge the target block and its buddy into a unified $2^{\{\text{order}+1\}}$ block

```
struct page* alloc_pages(unsigned int order) {  
    // TODO: Implement this function  
}  
  
void free_pages(struct page* page) {  
    // TODO: Implement this function  
}
```



Exercise 3-1: Expected Result

```
(base) hsien@hsien-System-Product-Name:~/OSC/lab3/ex1$ make run
```

```
./prog
```

```
p1:  
free_area[10] 2559  
free_area[9] 1  
free_area[8] 1  
free_area[7] 1  
free_area[6] 1  
free_area[5] 1  
free_area[4] 1  
free_area[3] 1  
free_area[2] 1  
free_area[1] 1  
free_area[0] 0
```

```
p2:  
free_area[10] 2559  
free_area[9] 1  
free_area[8] 1  
free_area[7] 1  
free_area[6] 1  
free_area[5] 1  
free_area[4] 1  
free_area[3] 1  
free_area[2] 1  
free_area[1] 0  
free_area[0] 0
```

```
p3:  
free_area[10] 2559  
free_area[9] 1  
free_area[8] 1  
free_area[7] 1  
free_area[6] 1  
free_area[5] 1  
free_area[4] 1  
free_area[3] 1  
free_area[2] 0  
free_area[1] 1  
free_area[0] 0
```

```
free:  
free_area[10] 2560  
free_area[9] 0  
free_area[8] 0  
free_area[7] 0  
free_area[6] 0  
free_area[5] 0  
free_area[4] 0  
free_area[3] 0  
free_area[2] 0  
free_area[1] 0  
free_area[0] 0
```



Exercise 3-1: Answer

```
struct page* alloc_pages(unsigned int order) {
    if (order > MAX_ORDER) return nullptr;

    for (unsigned int current_order = order; current_order <= MAX_ORDER; ++current_order) {
        if (!free_area[current_order].empty()) {
            struct page* page = free_area[current_order].front();
            free_area[current_order].pop_front();

            while (current_order > order) {
                --current_order;
                size_t pfn = page - mem_map.data();
                size_t buddy_pfn = pfn + (1 << current_order);
                struct page* buddy = &mem_map[buddy_pfn];

                buddy->order = current_order;
                buddy->refcount = 0;
                free_area[current_order].push_back(buddy);
            }

            page->order = order;
            page->refcount = 1;
            return page;
        }
    }
    return nullptr;
}
```



Exercise 3-1: Answer

```
void free_pages(struct page* page) {
    if (!page || page->refcount == 0) return;

    size_t pfn = page - mem_map.data();
    unsigned int order = page->order;
    page->refcount = 0;

    while (order < MAX_ORDER) {
        size_t buddy_pfn = pfn ^ (1 << order);
        if (buddy_pfn >= NUM_PAGES) break;

        struct page* buddy = &mem_map[buddy_pfn];

        if (buddy->refcount != 0 || buddy->order != order) {
            break;
        }

        free_area[order].remove(buddy);

        if (buddy_pfn < pfn) {
            page = buddy;
            pfn = buddy_pfn;
        }

        order++;
        page->order = order;
    }

    free_area[order].push_back(page);
}
```



Exercise 3-2: Memory Reservation

- Mark specific physical memory regions (e.g., Kernel Image, DTB) as occupied to prevent the Buddy System from allocating them
- Top-Down Splitting
 - **No Overlap** : Block is safe and keep it in the free list
 - **Full Overlap** : Block falls entirely within the reserved region (Remove from the free list and set “**refcount**” as allocated)
 - **Partial Overlap** : Split it into two equal child blocks, and push them to the next lower level (order - 1) for further evaluation



Exercise 3-2: Reserved Memory Regions

- The following memory regions must be reserved before any allocation:
 - Device Tree Blob (DTB)
 - Kernel image
 - Initramfs (ramdisk)
 - Any additional platform-specific reserved areas

```
void memory_reserve(phys_addr_t base, size_t size) {  
    // TODO: Implement this function  
}
```



Exercise 3-2: Expected Result

```
(base) hsien@hsien-System-Product-Name:~/OSC/lab3/ex2$ make run
rm -f prog
g++ -g -o prog main.cpp
./prog
free_area[10] 2037
free_area[9] 0
free_area[8] 1
free_area[7] 1
free_area[6] 0
free_area[5] 0
free_area[4] 1
free_area[3] 0
free_area[2] 1
free_area[1] 1
free_area[0] 0
```



Exercise 3-2: Answer

```
void memory_reserve(phys_addr_t base, size_t size) {
    if (size == 0) return;

    size_t start_pfn = base / PAGE_SIZE;
    size_t end_pfn = (base + size + PAGE_SIZE - 1) / PAGE_SIZE;

    for (int order = MAX_ORDER; order >= 0; --order) {
        auto it = free_area[order].begin();
        while (it != free_area[order].end()) {
            struct page* p = *it;
            size_t pfn = p - mem_map.data();
            size_t block_start = pfn;
            size_t block_end = pfn + (1 << order);

            if (block_end <= start_pfn || block_start >= end_pfn) {
                ++it;
                continue;
            }

            it = free_area[order].erase(it);

            if (block_start >= start_pfn && block_end <= end_pfn) {
                p->refcount = 1;
                continue;
            }

            int next_order = order - 1;
            struct page* buddy = &mem_map[pfn + (1 << next_order)];

            p->order = next_order;
            buddy->order = next_order;

            free_area[next_order].push_back(p);
            free_area[next_order].push_back(buddy);
        }
    }
}
```



TODOs

Exercise 3-1

- **alloc_pages** : Allocate a contiguous block of pages using the split logic
- **free_pages** : Release the specified page block and iteratively merge it with adjacent unallocated buddies

Exercise 3-2

- **memory_reserve** : Isolate specific physical memory regions from the allocator by marking their pages as allocated